DBMS INTERVIEW QUESTIONS

Important Questions.

What is a DBMS and how is it different from a file system? A Database Management System (DBMS) is software that allows users to store, manage, and retrieve data in a structured and efficient manner. It provides features like data security, concurrency control, and query processing using SQL. In contrast, a file system stores data in files without enforcing structure or relationships. DBMS reduces redundancy, ensures consistency, and supports multi-user access, whereas file systems are prone to duplication, inconsistency, and lack advanced querying capabilities.

What are the types of DBMS? DBMS can be categorized into hierarchical, network, relational, and object-oriented types. Hierarchical DBMS organizes data in a tree structure with parent-child relationships. Network DBMS allows multiple relationships forming a graph structure. Relational DBMS stores data in tables with rows and columns and is the most widely used. Object-oriented DBMS stores data as objects, similar to object-oriented programming, making it suitable for complex data types.

What is the difference between schema and instance? A schema represents the logical structure or design of a database, including tables, attributes, and relationships. It acts as a blueprint and remains relatively stable. An instance refers to the actual data stored in the database at a particular moment, which changes frequently as data is inserted, updated, or deleted.

What is a primary key and a foreign key? A primary key is an attribute or a set of attributes that uniquely identifies each record in a table and cannot contain null values. A foreign key is an attribute in one table that refers to the primary key of another table, establishing a relationship between the two tables and ensuring referential integrity.

What are constraints in DBMS? Constraints are rules applied to database columns to enforce data integrity. Common constraints include NOT NULL, which prevents null values; UNIQUE, which ensures all values are distinct; PRIMARY KEY, which uniquely identifies records; FOREIGN KEY, which maintains relationships between tables; CHECK, which enforces a condition on values; and DEFAULT, which assigns a default value when none is provided.

What is normalization? Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It involves dividing large tables into smaller ones and defining relationships between them. Different normal forms include 1NF, which ensures atomic values; 2NF, which removes partial dependency; 3NF, which removes transitive dependency; and BCNF, which is a stricter version of 3NF.

What is denormalization? Denormalization is the process of combining tables and introducing redundancy to improve read performance. It reduces the need for complex joins, making

queries faster, especially in read-heavy systems, but increases storage requirements and the risk of data inconsistency.

What are ACID properties? ACID properties ensure reliable transaction processing in a DBMS. Atomicity ensures that a transaction is executed completely or not at all. Consistency ensures that the database remains in a valid state before and after a transaction. Isolation ensures that concurrent transactions do not interfere with each other. Durability guarantees that once a transaction is committed, the changes are permanently stored.

What is a transaction? A transaction is a sequence of operations performed as a single logical unit of work. All operations within a transaction must be completed successfully; otherwise, the entire transaction is rolled back. Transactions are essential for maintaining data consistency, such as in banking operations where money is transferred between accounts.

What is a deadlock? A deadlock is a situation where two or more transactions are waiting for each other to release resources, causing all of them to be stuck indefinitely. This typically occurs when each transaction holds a resource that the other needs, preventing progress.

What are different types of joins? Joins are used to combine data from multiple tables based on related columns. An inner join returns only matching records from both tables. A left join returns all records from the left table and matching records from the right table. A right join returns all records from the right table and matching records from the left table. A full join returns all records from both tables, and a cross join returns the Cartesian product of both tables.

What is the difference between WHERE and HAVING? The WHERE clause is used to filter rows before grouping is applied, while the HAVING clause is used to filter groups after aggregation. WHERE cannot be used with aggregate functions directly, whereas HAVING is specifically used for conditions involving aggregate functions.

What is indexing? Indexing is a technique used to improve the speed of data retrieval operations by creating a data structure that allows quick lookup of records. It reduces the need for full table scans. However, indexing increases storage overhead and can slow down insert, update, and delete operations.

What is a view? A view is a virtual table created from a query. It does not store data physically but provides a way to present data in a specific format. Views are useful for simplifying complex queries and restricting access to sensitive data.

What is a stored procedure? A stored procedure is a precompiled set of SQL statements stored in the database. It can be executed repeatedly, improving performance and reducing network communication between the application and the database.

What is a trigger? A trigger is a set of SQL statements that automatically executes in response to specific events such as insert, update, or delete operations on a table. It is used to enforce business rules and maintain data integrity.

What is functional dependency? Functional dependency is a relationship between attributes where one attribute uniquely determines another. For example, if attribute A determines attribute B, then knowing the value of A allows you to determine the value of B.

What is a candidate key? A candidate key is a minimal set of attributes that can uniquely identify a record in a table. There can be multiple candidate keys in a table, and one of them is chosen as the primary key.

What is SQL? SQL, or Structured Query Language, is used to interact with relational databases. It is used to retrieve data using SELECT, insert data using INSERT, update data using UPDATE, delete data using DELETE, and define database structures using commands like CREATE and ALTER.

What is the difference between DELETE, TRUNCATE, and DROP? DELETE removes specific rows from a table and can be rolled back. TRUNCATE removes all rows from a table quickly and typically cannot be rolled back. DROP deletes the entire table structure along with its data permanently.

What is concurrency control? Concurrency control ensures that multiple transactions can execute simultaneously without compromising data integrity. It uses techniques such as locking, timestamp ordering, and multi-version concurrency control to maintain consistency.

What are isolation levels? Isolation levels define how transaction integrity is maintained when multiple transactions are executed concurrently. The levels include Read Uncommitted, Read Committed, Repeatable Read, and Serializable, with increasing levels of data protection and decreasing concurrency.

What are anomalies in DBMS? Anomalies are problems that occur due to poor database design. Insertion anomalies occur when data cannot be inserted without additional unrelated data. Deletion anomalies occur when deleting a record causes unintended loss of other data. Update anomalies occur when the same data must be updated in multiple places, increasing the risk of inconsistency.

What is the ER model? The Entity-Relationship model is used for designing databases. It represents data as entities, attributes, and relationships, helping in visualizing and structuring the database before implementation.

What is a B+ tree? A B+ tree is a balanced tree data structure used in database indexing. It stores all data records at the leaf nodes and uses internal nodes as indexes, enabling efficient searching, insertion, and deletion operations, especially for range queries.

Why is indexing not always beneficial? While indexing speeds up data retrieval, it adds overhead during data modification operations such as insert, update, and delete. Maintaining indexes requires additional storage and processing, which can degrade performance in write-heavy systems.

What is the difference between clustered and non-clustered indexes? A clustered index determines the physical order of data in a table, meaning the table data is stored in sorted order based on the index. A non-clustered index, on the other hand, maintains a separate structure with pointers to the actual data, allowing multiple indexes on a table.

Why is normalization up to 3NF usually sufficient? Normalization up to the Third Normal Form is usually sufficient because it removes most redundancy and dependency issues while maintaining a balance between performance and complexity. Higher normal forms provide marginal benefits but increase design complexity.

What happens if ACID properties are violated? If ACID properties are violated, the database may become inconsistent, leading to incorrect or unreliable data. Transactions may be partially executed, concurrent operations may interfere, and committed data may not be durable, resulting in data corruption and system failures.

1. Basics of DBMS

Q1. What is DBMS?
A Database Management System (DBMS) is software that enables users to store, retrieve, manage, and manipulate data efficiently. It acts as an interface between the database and users or applications, ensuring data consistency, security, and integrity. Examples include MySQL, Oracle Database, and Microsoft SQL Server.

Q2. What are the advantages of DBMS?
DBMS provides data abstraction, reduces redundancy, ensures data consistency, supports concurrent access, provides backup and recovery mechanisms, and enforces security constraints. It also allows multiple users to access data simultaneously without conflicts.

Q3. What is data abstraction?
Data abstraction is the process of hiding complex implementation details and showing only essential features to users. It is achieved through three levels: physical level (how data is stored), logical level (what data is stored), and view level (how users see the data).

---

2. ER Model

Q4. What is an ER model?
The Entity-Relationship (ER) model is a high-level conceptual data model used to represent real-world entities and their relationships. It consists of entities, attributes, and relationships.

Q5. What is the difference between entity and attribute?
An entity represents a real-world object such as a student or employee, while an attribute represents a property of that entity, such as name, age, or ID.

Q6. What are different types of attributes?
Attributes can be simple, composite, derived, or multi-valued. Simple attributes cannot be divided further, composite attributes can be broken into sub-parts, derived attributes are computed from other attributes, and multi-valued attributes can have multiple values.

---

3. Relational Model

Q7. What is a relation?
A relation is a table consisting of rows (tuples) and columns (attributes). Each relation represents a set of data with a specific structure.

Q8. What is a primary key?
A primary key is an attribute or set of attributes that uniquely identifies each tuple in a relation. It cannot have NULL values and must be unique.

Q9. What is a foreign key?
A foreign key is an attribute that refers to the primary key of another table. It is used to maintain referential integrity between tables.

---

4. Normalization

Q10. What is normalization?
Normalization is the process of organizing data to minimize redundancy and improve data integrity. It involves dividing tables into smaller tables and defining relationships.

Q11. What are different normal forms?
Normal forms include 1NF, 2NF, 3NF, BCNF, 4NF, and 5NF. Each level removes specific types of anomalies and redundancy.

Q12. What is 3NF?
A relation is in Third Normal Form if it is in 2NF and there is no transitive dependency, meaning non-prime attributes do not depend on other non-prime attributes.

Q13. What is BCNF?
Boyce-Codd Normal Form is a stricter version of 3NF where for every functional dependency, the determinant must be a candidate key.

---

5. SQL

Q14. What is SQL?
Structured Query Language (SQL) is used to interact with relational databases for querying, updating, and managing data.

Q15. What is the difference between DELETE, TRUNCATE, and DROP?
DELETE removes specific rows and can be rolled back. TRUNCATE removes all rows quickly and cannot be rolled back in most systems. DROP deletes the entire table structure along with data.

Q16. What is a JOIN?
A JOIN combines rows from two or more tables based on a related column. Types include INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.

Q17. What is a subquery?
A subquery is a query nested inside another query. It is used to perform operations that require multiple steps.

---

6. Transactions

Q18. What is a transaction?
A transaction is a sequence of operations performed as a single logical unit of work. It must satisfy ACID properties.

Q19. What are ACID properties?
ACID stands for Atomicity, Consistency, Isolation, and Durability. Atomicity ensures all operations complete or none do, consistency ensures valid data states, isolation ensures concurrent transactions do not interfere, and durability ensures changes are permanent.

---

7. Concurrency Control

Q20. What is concurrency control?
Concurrency control ensures that multiple transactions can execute simultaneously without leading to inconsistency.

Q21. What is a deadlock?
A deadlock occurs when two or more transactions wait indefinitely for each other to release locks.

Q22. What is locking?
Locking is a mechanism to control concurrent access to data. Shared locks allow reading, while exclusive locks allow writing.

---

8. Indexing

Q23. What is an index?
An index is a data structure that improves the speed of data retrieval operations at the cost of additional storage.

Q24. What is the difference between clustered and non-clustered index?
A clustered index determines the physical order of data in a table, whereas a non-clustered index creates a separate structure to point to the data.

---

9. File Organization

Q25. What are different file organizations?
File organizations include sequential, heap, and hashed file organization. Each method determines how records are stored and accessed.

---

10. DBMS Architecture

Q26. What is three-schema architecture?
The three-schema architecture consists of external, conceptual, and internal schemas, which provide abstraction and independence.

---

11. Keys

Q27. What are different types of keys?
Keys include primary key, candidate key, super key, alternate key, and foreign key. Each serves a specific purpose in uniquely identifying records.

---

12. Views

Q28. What is a view?
A view is a virtual table based on a SQL query. It does not store data physically but displays data from one or more tables.

---

13. Stored Procedures and Triggers

Q29. What is a stored procedure?
A stored procedure is a precompiled set of SQL statements stored in the database, used to perform repetitive tasks efficiently.

Q30. What is a trigger?
A trigger is a procedure that automatically executes in response to specific events such as INSERT, UPDATE, or DELETE.

---

14. NoSQL vs SQL

Q31. What is the difference between SQL and NoSQL?
SQL databases are relational and use structured schemas, while NoSQL databases are non-relational, schema-less, and designed for scalability and flexibility.

---

15. Miscellaneous Important Questions

Q32. What is data independence?
Data independence is the ability to change the schema without affecting applications. It includes logical and physical independence.

Q33. What is denormalization?
Denormalization is the process of combining tables to reduce joins and improve read performance, at the cost of redundancy.

Q34. What are anomalies in DBMS?
Anomalies include insertion, deletion, and update anomalies, which occur due to poor database design.

16. Functional Dependency & Theory

Q35. What is a functional dependency?
A functional dependency (FD) is a relationship between two attributes in a relation, where one attribute uniquely determines another. It is denoted as $X \rightarrow Y$, meaning if two tuples have the same value for X, they must have the same value for Y. Functional dependencies are the foundation of normalization.

Q36. What is a trivial functional dependency?
A functional dependency is trivial if the right-hand side is a subset of the left-hand side. For example, $\{A, B\} \rightarrow A$ is trivial because A is already part of the determinant.

Q37. What is closure of attributes?
The closure of a set of attributes is the set of all attributes that can be functionally determined from it. It is used to find candidate keys and check normalization.

Q38. What is a candidate key?
A candidate key is a minimal set of attributes that can uniquely identify a tuple. There can be multiple candidate keys in a table, and one of them is selected as the primary key.

---

17. Advanced Normalization

Q39. What is 2NF?
A relation is in Second Normal Form if it is in 1NF and has no partial dependency, meaning no non-prime attribute depends on a subset of a candidate key.

Q40. What is 4NF?
A relation is in Fourth Normal Form if it is in BCNF and has no multi-valued dependencies. This ensures that independent multi-valued attributes are stored separately.

Q41. What is 5NF?
Fifth Normal Form deals with join dependencies. A relation is in 5NF if it cannot be decomposed further without losing information.

---

18. Query Optimization

Q42. What is query optimization?
Query optimization is the process of selecting the most efficient execution plan for a query. The DBMS considers multiple strategies and chooses the one with the lowest cost in terms of time and resources.

Q43. What is a query execution plan?
A query execution plan is a sequence of steps used by the DBMS to execute a query, including scanning tables, using indexes, and performing joins.

---

19. Transactions (Advanced)

Q44. What is a schedule?
A schedule is the order in which operations of multiple transactions are executed. It determines how transactions interleave.

Q45. What is serializability?
Serializability ensures that a concurrent schedule produces the same result as some serial execution of transactions, preserving consistency.

Q46. What is conflict serializability?
A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

---

20. Concurrency Problems

Q47. What is a dirty read?
A dirty read occurs when a transaction reads data that has been modified by another transaction but not yet committed.

Q48. What is a non-repeatable read?
A non-repeatable read happens when a transaction reads the same row twice and gets different values because another transaction modified it.

Q49. What is a phantom read?
A phantom read occurs when new rows are inserted or deleted by another transaction, affecting the result of a query.

---

21. Isolation Levels

Q50. What are isolation levels in DBMS?
Isolation levels define the degree to which transactions are isolated. The levels are Read Uncommitted, Read Committed, Repeatable Read, and Serializable, each preventing different types of anomalies.

---

22. Indexing (Advanced)

Q51. What is B+ Tree index?
A B+ Tree is a balanced tree data structure used in DBMS indexing. It allows efficient searching, insertion, and deletion operations and stores all records at the leaf level.

Q52. What is hashing in DBMS?
Hashing uses a hash function to map keys to specific locations in memory, enabling fast data retrieval.

---

23. Storage & File Structure

Q53. What is heap file organization?
Heap file organization stores records in no particular order. It allows fast insertion but slower search operations.

Q54. What is sequential file organization?
In sequential organization, records are stored in sorted order, which improves search performance but slows down insertion.

---

24. Distributed DBMS

Q55. What is distributed DBMS?
A distributed DBMS manages databases distributed across multiple locations while appearing as a single database to users.

Q56. What is fragmentation?
Fragmentation divides a database into smaller parts, which can be horizontal, vertical, or mixed, to improve performance and scalability.

---

25. CAP Theorem

Q57. What is CAP theorem?
The CAP theorem states that a distributed system can provide only two out of three guarantees: Consistency, Availability, and Partition Tolerance.

---

26. RAID

Q58. What is RAID?
RAID (Redundant Array of Independent Disks) is a storage technology used to improve performance and reliability through redundancy.

---

27. Views & Materialized Views

Q59. What is a materialized view?
A materialized view stores the result of a query physically, unlike a normal view. It improves performance for complex queries.

---

28. Triggers & Procedures (Advanced)

Q60. What are advantages of stored procedures?
Stored procedures improve performance, reduce network traffic, and enhance security by encapsulating logic inside the database.

Q61. What are disadvantages of triggers?
Triggers can increase complexity, make debugging difficult, and may degrade performance if overused.

---

29. NoSQL Deep Dive

Q62. What are types of NoSQL databases?
Types include key-value stores, document databases, column-family stores, and graph databases.

---

30. Real-World Scenario Questions (VERY IMPORTANT)

Q63. How would you design a database for a banking system?
A banking system requires tables for customers, accounts, transactions, and branches. Relationships must ensure referential integrity, and transactions must follow strict ACID properties to maintain financial accuracy.

Q64. How do you handle large-scale data in DBMS?
Large-scale data is handled using indexing, partitioning, sharding, caching, and distributed databases.

Q65. How do you improve query performance?
Query performance can be improved by using indexes, avoiding unnecessary joins, optimizing queries, and analyzing execution plans.

---

31. Frequently Asked Conceptual Questions

Q66. What is difference between DBMS and RDBMS?
DBMS stores data in files, while RDBMS uses tables with relationships and enforces constraints.

Q67. What is schema vs instance?
Schema is the structure of the database, while instance is the actual data at a given moment.

Q68. What is cardinality?
Cardinality defines the number of relationships between entities, such as one-to-one, one-to-many, or many-to-many.

---

32. Joins (Advanced)

Q69. What is self join?
A self join is when a table is joined with itself to compare rows within the same table.

Q70. What is cross join?
A cross join returns the Cartesian product of two tables.

---

33. Advanced Interview Traps

Q71. Why is normalization not always preferred?
Normalization can increase the number of joins required, which may degrade performance in read-heavy systems.

Q72. What is difference between WHERE and HAVING?
WHERE filters rows before grouping, while HAVING filters groups after aggregation.

---

34. Locks & Protocols

Q73. What is two-phase locking?
Two-phase locking ensures serializability by dividing execution into a growing phase (acquiring locks) and a shrinking phase (releasing locks).

---

35. Recovery

Q74. What is log-based recovery?
Log-based recovery uses logs to restore the database after a failure by replaying or undoing transactions.

Q75. What is checkpointing?
Checkpointing saves the current state of the database to reduce recovery time after a crash.

SQL INTERVIEW QUESTIONS

1. SQL Basics

Q1. What is SQL?
SQL (Structured Query Language) is a standard language used to interact with relational databases. It is used for storing, retrieving, updating, and deleting data. SQL is supported by systems like MySQL, PostgreSQL, and Oracle Database.

Q2. What are different types of SQL commands?
SQL commands are categorized into DDL, DML, DCL, and TCL. DDL (Data Definition Language) includes commands like CREATE and DROP, DML (Data Manipulation Language) includes INSERT and UPDATE, DCL (Data Control Language) includes GRANT and REVOKE, and TCL (Transaction Control Language) includes COMMIT and ROLLBACK.

---

2. Data Definition Language (DDL)

Q3. What is CREATE command?
CREATE is used to create database objects such as tables, views, and indexes. It defines the structure of the database.

Q4. What is ALTER command?
ALTER is used to modify an existing table structure, such as adding or removing columns or changing data types.

Q5. What is DROP vs TRUNCATE?
DROP deletes the entire table along with its structure, whereas TRUNCATE removes all rows but keeps the table structure intact. TRUNCATE is faster because it does not log individual row deletions.

---

3. Data Manipulation Language (DML)

Q6. What is INSERT command?
INSERT is used to add new records into a table. It can insert single or multiple rows at once.

Q7. What is UPDATE command?
UPDATE modifies existing records in a table based on a condition specified using the WHERE clause.

Q8. What is DELETE command?
DELETE removes specific rows from a table based on conditions. It can be rolled back if used within a transaction.

---

4. Constraints

Q9. What are constraints in SQL?
Constraints are rules applied on table columns to enforce data integrity. Examples include NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, and CHECK.

Q10. What is PRIMARY KEY constraint?
A PRIMARY KEY uniquely identifies each record in a table and does not allow NULL values.

Q11. What is FOREIGN KEY constraint?
A FOREIGN KEY establishes a relationship between two tables and ensures referential integrity.

---

5. SELECT Queries

Q12. What is SELECT statement?
SELECT is used to retrieve data from a database. It allows filtering, sorting, and grouping of data.

Q13. What is WHERE clause?
WHERE filters rows based on conditions before returning results.

Q14. What is ORDER BY?
ORDER BY sorts the result set in ascending or descending order.

---

6. Aggregate Functions

Q15. What are aggregate functions?
Aggregate functions perform calculations on a set of values and return a single result. Examples include COUNT, SUM, AVG, MIN, and MAX.

Q16. What is GROUP BY?
GROUP BY groups rows with similar values into summary rows for aggregation.

Q17. What is HAVING clause?
HAVING is used to filter grouped data after aggregation.

---

7. Joins

Q18. What is JOIN in SQL?
JOIN is used to combine rows from multiple tables based on related columns.

Q19. What are types of JOINs?
Types include INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN. INNER JOIN returns matching rows, while LEFT and RIGHT JOIN include unmatched rows from one side.

Q20. What is SELF JOIN?
SELF JOIN is when a table is joined with itself to compare rows within the same table.

---

8. Subqueries

Q21. What is a subquery?
A subquery is a query nested inside another query, used for complex filtering or calculations.

Q22. What is correlated subquery?
A correlated subquery executes once for each row of the outer query, making it slower but useful for row-by-row comparisons.

---

9. Indexing

Q23. What is an index in SQL?
An index is a data structure that improves query performance by allowing faster data retrieval.

Q24. What are advantages and disadvantages of indexing?
Indexes speed up read operations but slow down write operations and consume extra storage.

---

10. Views

Q25. What is a view?
A view is a virtual table created from a query. It does not store data physically.

Q26. What is materialized view?
A materialized view stores query results physically, improving performance for repeated queries.

---

11. Transactions

Q27. What is a transaction in SQL?
A transaction is a group of SQL operations executed as a single unit. It ensures data integrity using ACID properties.

Q28. What is COMMIT and ROLLBACK?
COMMIT saves changes permanently, while ROLLBACK undoes changes since the last commit.

---

12. Window Functions (VERY IMPORTANT)

Q29. What are window functions?
Window functions perform calculations across a set of rows related to the current row without collapsing the result set.

Q30. What is ROW_NUMBER()?
ROW_NUMBER assigns a unique number to each row within a partition.

Q31. What is RANK() vs DENSE_RANK()?
RANK skips numbers in case of ties, while DENSE_RANK does not.

---

13. Advanced SQL Concepts

Q32. What is CTE (Common Table Expression)?
A CTE is a temporary result set defined using the WITH clause, improving readability of complex queries.

Q33. What is difference between CTE and subquery?
CTEs are more readable and reusable, while subqueries are nested directly inside queries.

---

14. Case-Based Questions (VERY IMPORTANT)

Q34. How to find second highest salary?
The second highest salary can be found using subqueries or window functions like RANK or DENSE_RANK.

Q35. How to remove duplicate rows?
Duplicates can be removed using DISTINCT, GROUP BY, or ROW_NUMBER with partitioning.

Q36. How to find employees without managers?
This can be solved using LEFT JOIN where manager ID is NULL.

---

15. Performance Optimization

Q37. How to optimize SQL queries?
Optimization includes using indexes, avoiding SELECT *, minimizing joins, and analyzing execution plans.

---

16. Advanced Joins & Queries

Q38. What is CROSS JOIN?
CROSS JOIN returns the Cartesian product of two tables.

Q39. What is NATURAL JOIN?
NATURAL JOIN automatically joins tables based on columns with the same name.

---

17. NULL Handling

Q40. How does SQL handle NULL values?
NULL represents missing or unknown data. Comparisons with NULL require IS NULL or IS NOT NULL.

---

18. Stored Procedures & Triggers

Q41. What is a stored procedure?
A stored procedure is a precompiled SQL code stored in the database for reuse.

Q42. What is a trigger?
A trigger automatically executes when a specified event occurs in a table.

---

19. Real Interview Scenarios

Q43. How to find nth highest salary?
This can be solved using window functions like DENSE_RANK or LIMIT with OFFSET.

Q44. How to find duplicate records?
Duplicates can be identified using GROUP BY and HAVING COUNT(*) > 1.

Q45. How to pivot data in SQL?
Pivoting transforms rows into columns using CASE statements or PIVOT functions.

---

20. Tricky Interview Questions

Q46. Difference between WHERE and HAVING?
WHERE filters rows before aggregation, while HAVING filters after aggregation.

Q47. Difference between UNION and UNION ALL?
UNION removes duplicates, while UNION ALL keeps all rows.

---

21. Practical Coding-Oriented Questions

Q48. Write query to find employees with salary greater than average.
This can be solved using a subquery that calculates average salary and compares each employee's salary.

Q49. Write query to get department-wise highest salary.
Use GROUP BY with MAX or window functions like RANK.

---

22. Advanced Topics (High-Level)

Q50. What is sharding?
Sharding distributes data across multiple servers to handle large-scale applications.

Q51. What is partitioning?
Partitioning divides a table into smaller parts to improve performance.

Tables used:

- `employees(emp_id, name, salary, dept_id, manager_id, hire_date)`
- `departments(dept_id, dept_name)`

1–10: Basic Queries

Q1. Select all employees

SELECT * FROM employees;

Q2. Select specific columns

SELECT name, salary FROM employees;

Q3. Filter employees with salary > 50000

SELECT * FROM employees

WHERE salary > 50000;

Q4. Employees in a specific department

SELECT * FROM employees

WHERE dept_id = 2;

Q5. Sort employees by salary descending

SELECT * FROM employees

ORDER BY salary DESC;

Q6. Count total employees

SELECT COUNT(*) FROM employees;

Q7. Find average salary

SELECT AVG(salary) FROM employees;

Q8. Find max salary

SELECT MAX(salary) FROM employees;

Q9. Find min salary

SELECT MIN(salary) FROM employees;

Q10. Distinct departments

SELECT DISTINCT dept_id FROM employees;

11–20: Aggregation & Grouping

Q11. Department-wise employee count

SELECT dept_id, COUNT(*)

FROM employees

GROUP BY dept_id;

Q12. Department-wise average salary

SELECT dept_id, AVG(salary)

FROM employees

GROUP BY dept_id;

Q13. Departments with more than 5 employees

SELECT dept_id, COUNT(*)

FROM employees

```
GROUP BY dept_id
HAVING COUNT(*) > 5;
```

Q14. Total salary per department

```
SELECT dept_id, SUM(salary)
FROM employees
GROUP BY dept_id;
```

Q15. Highest salary in each department

```
SELECT dept_id, MAX(salary)
FROM employees
GROUP BY dept_id;
```

Q16. Departments where avg salary > 60000

```
SELECT dept_id, AVG(salary)
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 60000;
```

Q17. Count employees hired after 2020

```
SELECT COUNT(*)
FROM employees
WHERE hire_date > '2020-01-01';
```

Q18. Sum of salaries for dept 1

```
SELECT SUM(salary)
FROM employees
WHERE dept_id = 1;
```

Q19. Find duplicate salaries

```
SELECT salary, COUNT(*)
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1;
```

Q20. Total employees per salary

```
SELECT salary, COUNT(*)
FROM employees
GROUP BY salary;
```

21–30: Joins

Q21. Inner Join employees with departments

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

Q22. Left Join

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

Q23. Right Join

```
SELECT e.name, d.dept_name
FROM employees e
```

```
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

Q24. Employees without department

```
SELECT e.name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Q25. Self Join (employee-manager)

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id;
```

Q26. Employees in same department

```
SELECT e1.name, e2.name
FROM employees e1
JOIN employees e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id != e2.emp_id;
```

Q27. Cross Join

```
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

Q28. Count employees per department (with join)

```
SELECT d.dept_name, COUNT(e.emp_id)
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Q29. Departments with no employees

```
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

Q30. Highest paid employee per department

```
SELECT e.*
FROM employees e
JOIN (
  SELECT dept_id, MAX(salary) AS max_sal
  FROM employees
  GROUP BY dept_id
) t
ON e.dept_id = t.dept_id AND e.salary = t.max_sal;
```

31–40: Subqueries & Advanced

Q31. Employees earning above average

```
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q32. Second highest salary

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q33. Nth highest salary (3rd)

```
SELECT DISTINCT salary
FROM employees e1
WHERE 3 = (
  SELECT COUNT(DISTINCT salary)
  FROM employees e2
  WHERE e2.salary >= e1.salary
);
```

Q34. Employees in departments with avg salary > 60000

```
SELECT *
FROM employees
WHERE dept_id IN (
  SELECT dept_id
  FROM employees
  GROUP BY dept_id
  HAVING AVG(salary) > 60000
);
```

Q35. Exists example

```
SELECT name
FROM employees e
WHERE EXISTS (
  SELECT 1
  FROM departments d
  WHERE d.dept_id = e.dept_id
);
```

Q36. Not Exists example

```
SELECT name
FROM employees e
WHERE NOT EXISTS (
  SELECT 1
  FROM departments d
  WHERE d.dept_id = e.dept_id
);
```

Q37. Employees with max salary overall

```
SELECT *
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Q38. Employees hired before manager

```
SELECT e.name
FROM employees e
JOIN employees m
```

```
ON e.manager_id = m.emp_id
WHERE e.hire_date < m.hire_date;
```

Q39. Find employees with same salary

```
SELECT e1.name, e2.name
FROM employees e1
JOIN employees e2
ON e1.salary = e2.salary
AND e1.emp_id != e2.emp_id;
```

Q40. Delete duplicate rows

```
DELETE FROM employees
WHERE emp_id NOT IN (
  SELECT MIN(emp_id)
  FROM employees
  GROUP BY name, salary
);
```

41–50: Window Functions (VERY IMPORTANT)

Q41. Row number

```
SELECT name, salary,
ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn
FROM employees;
```

Q42. Rank

```
SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees;
```

Q43. Dense Rank

```
SELECT name, salary,
DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees;
```

Q44. Top 3 salaries

```
SELECT *
FROM (
  SELECT *, DENSE_RANK() OVER (ORDER BY salary DESC) rnk
  FROM employees
) t
WHERE rnk <= 3;
```

Q45. Partition by department

```
SELECT name, dept_id, salary,
RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) rnk
FROM employees;
```

Q46. Running total salary

```
SELECT name, salary,
SUM(salary) OVER (ORDER BY emp_id) AS running_total
FROM employees;
```

Q47. Moving average

```
SELECT name, salary,
```

AVG(salary) OVER (ORDER BY emp_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)

FROM employees;

Q48. Lead function

SELECT name, salary,

LEAD(salary) OVER (ORDER BY salary) AS next_salary

FROM employees;

Q49. Lag function

SELECT name, salary,

LAG(salary) OVER (ORDER BY salary) AS prev_salary

FROM employees;

Q50. Find gap between salaries

SELECT name, salary,

salary - LAG(salary) OVER (ORDER BY salary) AS diff

FROM employees;